



CS450 Project Part 1

Edge-Cloud Continuum Speech to Text and
Speaker Diarization Project

APR 2026

Ata Tan Dağdır - S028835



Introduction & Background

Mimi (Mimi: AI Meeting Notes on App Store) is an AI-powered speech transcription application built with React Native. Currently it relies on public cloud APIs (like Google Cloud STT). To improve performance, reduce scaling costs, strengthen privacy and to provide offline support we are migrating to a Cloud-Edge Continuum architecture.

This means deploying STT and speaker diarization models locally on mobile edge devices and our servers, dynamically routing the workload between them based on real-time conditions.

Our solution will involve several parts:

1. **Edge Processing:** Deploying the speech models onto the user's devices, this will constrain us heavily on sizes and processing powers of the edge models.
2. **Cloud Processing:** The backend will handle heavy-duty diarization tasks, premium users' requests and low confidence transcriptions.
3. **Orchestrator:** The system will automatically decide whether to process audio on the phone or the cloud based on network connectivity, battery life, device power, user status and transcription confidence.



Literature Review

Recent advancements in mobile computing have shifted incredibly on how AI/ML workloads are distributed across systems. Traditionally, computationally intensive tasks like Automatic Speech Recognition (ASR) and speaker diarization were strictly done on the cloud. However, as user demand for real-time, private transcription has increased significantly, researchers have deduced significant latency, cost, and security issues in cloud-based implementations. Traditional cloud computing is fundamentally limited by latency and bandwidth constraints, making it inefficient for processing large and real-time data streams (Xia et al., 2021).

To address these limitations, recent literature points to a transition toward Edge AI. Jooluri et al. (2025) demonstrated that deploying optimized acoustic models directly onto mobile edge devices significantly reduces latency compared to traditional cloud infrastructure, while also providing data privacy by processing data locally.

However, mobile devices of 2026 are constrained by battery life and processing power, meaning they cannot completely replace the cloud for computationally heavy tasks (Zhou et al., 2025). Consequently, current research advocates for a Cloud-Edge Continuum method. Khalyeyev et al. (2023) define this as a unified computational hierarchy that treats mobile devices as nodes actively involved in the processing rather than only data providers. Furthermore, Al-Shaboti et al. (2022) established that the most efficient way to manage this continuum is through dynamic computation offloading. Basically, utilizing an orchestrator to route tasks based on real-time network and device conditions, a load-manager of sorts. This project directly applies these established frameworks to optimize the Mimi application.



Project Plan & Implementation

To migrate the Mimi to a Cloud-Edge Continuum, the project will be executed in three phases. Deploying edge models, configuring the cloud backend and developing the Orchestrator.

Phase 1: Edge Models & Deployment

Constraints on picking the edge models are memory, thermal throttling and storage. Therefore we have to use lightweight, heavily quantized models that can run locally.

- **Edge Speech To Text Models:** We will be evaluating Whisper.cpp, Moonshine and Vosk models for STT. Compared to Whisper.cpp, Moonshine uses significantly less memory and compute while providing competitive accuracy (King et al., 2025). Additionally, Vosk is included to evaluate ultra-low latency, real-time streaming performance on highly constrained hardware (Graham et al., 2025).
- **Edge Speaker Diarization Models:** Compared to STT, diarization is a much more challenging task at the present moment due to the heavy memory overhead. But fortunately, there are new state-of-the-art models we can use for this. We have two options. First one is Picovoice Falcon, a cross-platform solution focused on constrained hardware (Picovoice, 2026). The second one is SpeakerKit, known for being an ultra-lightweight framework that achieves high-speed inference. (Pacheco et al., 2025). Even though our app is currently only on IOS, we plan to release it for Android on 15th of April 2026, therefore we need a cross-platform solution. Since the android SDK for SpeakerKit was released on March 18, 2026, we can't decide on these models before evaluating them in our system.

Phase 2: Cloud Models

The cloud server will act as the fallback for heavy-duty computation. It will deploy heavier models for better accuracy. We could also pivot to using public cloud APIs for certain tasks. That will be decided after the project sets its course.

- **Cloud Speech To Text Models:** For models we can use on the cloud, we will evaluate STT APIs and models we can host on our own machines. For managed APIs, we will test Deepgram for its ultra-low latency (Backman, 2026) and AssemblyAI for asynchronous accuracy (AssemblyAI, 2024). To evaluate a self-hosted architecture we will use the Whisper Large-v3 model (Radford et al., 2022) deployed on our own cloud infrastructure. We will also compare these models to our existing Google Cloud STT & Diarization API.
- **Cloud Speaker Diarization Models:** For our managed cloud option, we will evaluate Deepgram's Nova-3, which integrates high-accuracy diarization directly into its ultra-low latency transcription pipeline (Backman, 2026). For our self-hosted architecture, we will evaluate Pyannote.audio, which remains the industry standard, a PyTorch based speaker diarization model (Bredin et al., 2020). We will also be comparing the results to the existing Google Cloud STT & Diarization API.

Phase 3: Edge Models & Deployment

The orchestrator will be built into the React Native app state manager. Before processing any audio, it will evaluate these metrics:

- Network Situation
- Device State (battery and low power mode)
- Task Confidence
- Device Specs
- User Status (premium users)

And the orchestrator will route the processing based on the decision made by this evaluation.

Note: This project will be funded by Ata Tan DAĞIDIR of Fidelio Software LLC.



Project Initiation Progress

Before building the Orchestrator, we must establish a baseline of how these models actually perform on mobile hardware. We will build a sandbox React Native application to run tests on physical devices (iPhone 15 and Motorola G03).

Here are some screenshots from our progress so far:

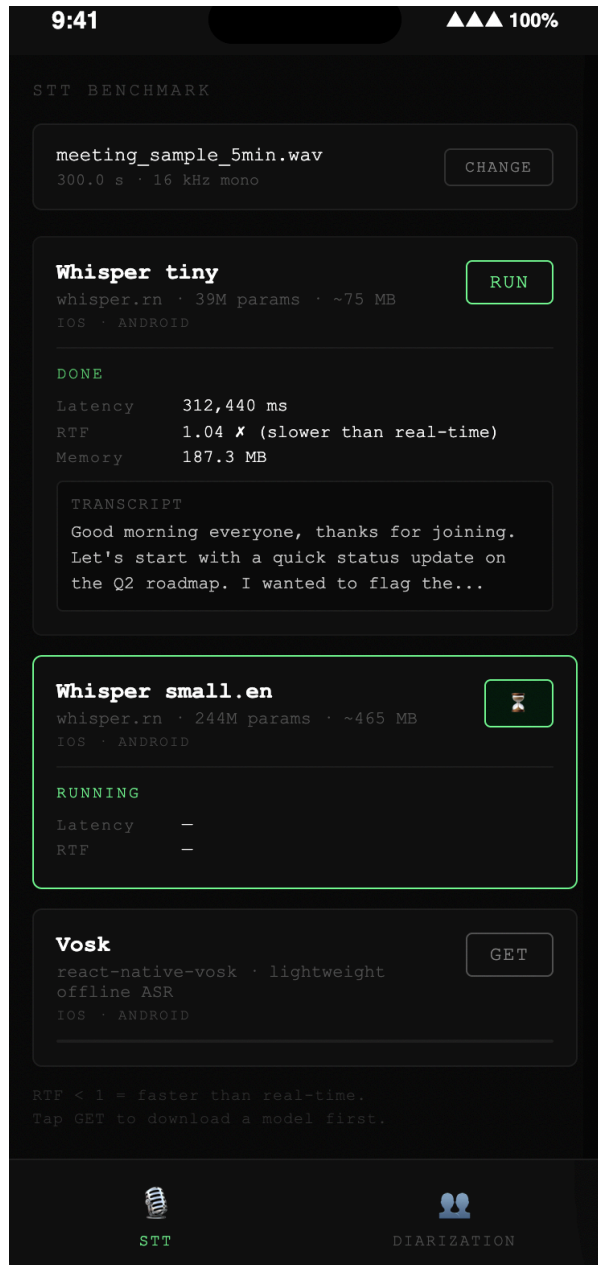


Figure 1: React Native UI for STT evaluation

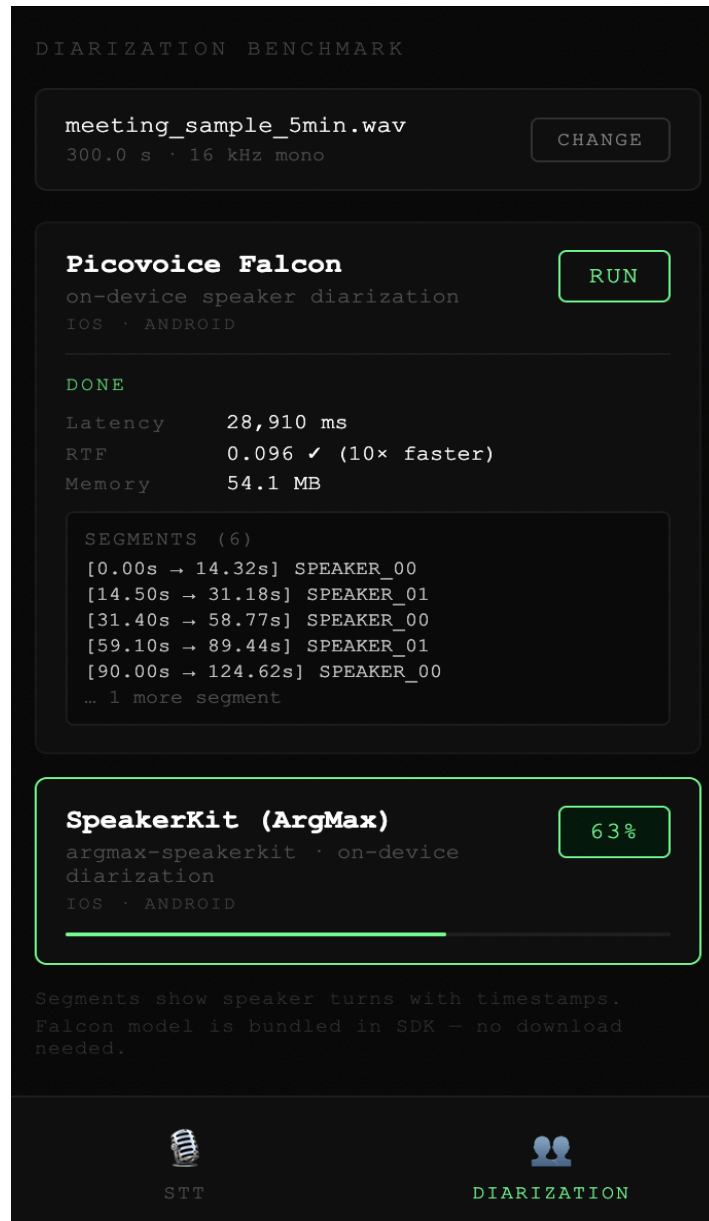


Figure 2: React Native UI for Diarization evaluation



References

- Al-Shaboti, M., et al. (2022). Mobility-Aware Computation Offloading in Edge Computing using Machine Learning. *IEEE Transactions on Mobile Computing*.
- Jooluri, N., et al. (2025). An Edge–Cloud Collaborative Architecture for Real-Time Speech Recognition Using MobileNet and Conformer. *3rd International Conference on Data Science and Network Security*.
- Khalyeyev, D., Bureš, T., & Hnětynka, P. (2023). Towards characterization of edge-cloud continuum. In *Software Architecture. ECSA 2022 Tracks and Workshops* (Vol. 13576, pp. 111–118). Springer Nature Switzerland.
- Xia, Q., Tao, Z., & Li, Q. (2021). Privacy Issues in Edge Computing. In *Fog/Edge Computing For Security, Privacy, and Applications* (pp. 147-167). Springer.
- Zhou, et al. (2025). Intelligent data analysis in edge computing with large language models: applications, challenges, and future directions. *Frontiers in Computer Science*.
- King, E., Sabra, A., & Warden, P. (2025). Flavors of Moonshine: Tiny specialized ASR models for edge devices. arXiv. <https://doi.org/10.48550/arXiv.2509.02523>
- Graham, O., et al. (2025). Comparative analysis of Vosk toolkit and other speech recognition frameworks for custom language model implementation. [Preprints.org](https://arxiv.org/abs/2509.02523).
- Pacheco, E., Orhon, A., Durmus, B., Munyampirwa, B., & Leonov, A. (2025). SDBench: A comprehensive benchmark suite for speaker diarization. arXiv. <https://doi.org/10.48550/arXiv.2507.16136>
- Picovoice. (2026). State of speaker diarization in 2026: pyannote vs Falcon benchmark. Picovoice. <https://picovoice.ai/blog/state-of-speaker-diarization/>

AssemblyAI. (2024). Universal-1: Robust and accurate multilingual speech-to-text. AssemblyAI Research. <https://www.assemblyai.com/research/universal-1>

Backman, M. (2026). Benchmarking STT for voice agents. Daily.co. <https://www.daily.co/blog/benchmarking-stt-for-voice-agents/>

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2022). Robust speech recognition via large-scale weak supervision. arXiv. <https://doi.org/10.48550/arXiv.2212.04356>

Bredin, H., Yin, R., Coria, J. M., Gelly, G., Korshunov, P., Lavechin, M., Fustes, D., Titeux, H., Bouaziz, W., & Gill, M. P. (2020). pyannote.audio: neural building blocks for speaker diarization. In ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP) (pp. 7124-7128). IEEE. <https://doi.org/10.1109/ICASSP40776.2020.9052974>